

TOWARDS MULTIMODAL TIME SERIES ANOMALY DETECTION WITH SEMANTIC ALIGNMENT AND CONDENSED INTERACTION

迈向基于语义对齐与紧凑交互的多模态时间序列异常检测

ICLR 2026



Chenjuan Guo

Professor, East China Normal University
在 dase.ecnu.edu.cn 的电子邮件经过验证
Data Analytics Machine Learning

关注

创建我的个人资料

标题	引用次数	年份
MoST: A Foundation Model for Multi-modality Spatio-temporal Traffic Prediction R Xu, J Chen, J Tian, C Guo, B Yang Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and ...	1	2026
Learning to factorize spatio-temporal foundation models S Zhong, J Qiu, Y Wu, X Zou, Z Rao, B Yang, C Guo, H Xu, Y Liang Advances in Neural Information Processing Systems 38, 88232-88260	3	2026
Dbloss: Decomposition-based loss function for time series forecasting X Qiu, X Wu, H Cheng, X Liu, C Guo, J Hu, B Yang Advances in Neural Information Processing Systems 38, 27741-27768	43	2026
Enhancing time series forecasting through selective representation spaces: A patch perspective X Wu, X Qiu, H Cheng, Z Li, J Hu, C Guo, B Yang Advances in Neural Information Processing Systems 38, 23328-23354	32	2026
CrossAD: Time series anomaly detection with cross-scale associations and cross-window modeling B Li, Q Shentu, Y Shu, H Zhang, M Li, N Jin, B Yang, C Guo Advances in Neural Information Processing Systems 38, 137959-137985	6	2026
Towards Multimodal Time Series Anomaly Detection with Semantic Alignment and Condensed		2026



1 这是什么领域

- 这是多模态时间序列异常检测领域，具体关注如何将时间序列模态与文本模态结合，用于识别时间序列中的异常事件。
- 文章不试图构建同时处理图像、视频等所有模态的通用多模态框架，而是聚焦于时间序列 + 文本这两类模态之间的语义对齐与交互。

2 问题公式化

给定长度为 T 的输入时间序列： $X = (x_1, \dots, x_T) \in \mathbb{R}^{T \times D}$ ，其中 D 是特征维度。

传统单模态时间序列异常检测需要输出： $Y = (y_1, \dots, y_T) \in \{0, 1\}^T$ ，其中 $y_t = 1$ 表示时间戳 t 被识别为异常。

在多模态时间序列异常检测任务中，除了时间序列 X ，还引入额外的文本模态。文章具体考虑将时间序列模态 X 与外生文本模态融合，其中外生文本表示为长度为 S 的序列： $C = (c_1, \dots, c_S) \in \mathbb{R}^S$ 。目标仍然是判断每个时间戳 t 是否异常，即确定 y_t 是正常还是异常。

挑战 1: 如何在异构多模态数据间实现语义一致对齐

大多数现有时间序列异常检测模型仍然局限在单模态数值框架中，忽视了利用多模态数据的潜力。

希望如何解决：通过**跨视角文本融合**与**多模态对齐**机制，融合**内源与外源**文本信息，实现时间与文本模式的语义一致对齐。

内源文本：用大模型从时序生成的衍生文本，虽天然对齐但语义贫乏，只能捕捉时序内在模式。

外源文本：引入新闻、文档作为外部上下文，但信息分散，难以与时序语义对齐。

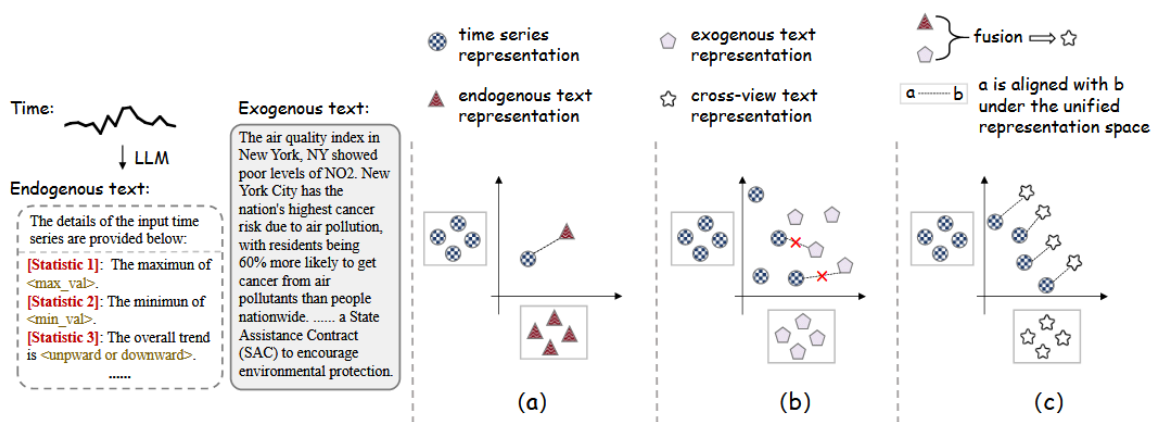


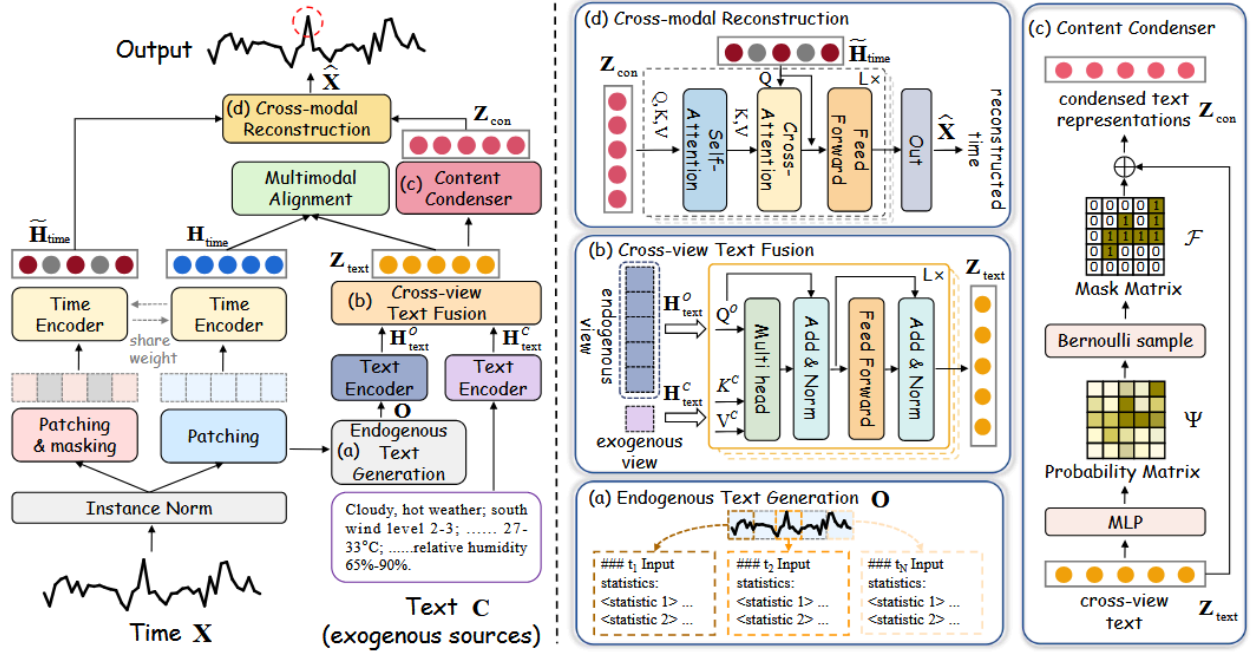
Figure 1: (a) LLM-based methods generate endogenous text from time series without incorporating exogenous information. (b) Exogenous-based methods incorporate text information by retrieving background knowledge from the web. The absence of connecting lines indicates that the two modalities are not aligned. (c) MindTS employs cross-view fusion to ensure semantic consistency between the exogenous text and the time series, enabling more precise alignment across modalities.

挑战 2：如何过滤冗余模态信息以有效增强跨模态交互

现有多模态时间序列方法通常采用**直接融合策略**，假设所有文本信息都是同等有用的。这种做法忽视了文本模态中可能存在的冗长细节或无关描述，这些内容可能会削弱真正有信息量内容的贡献。

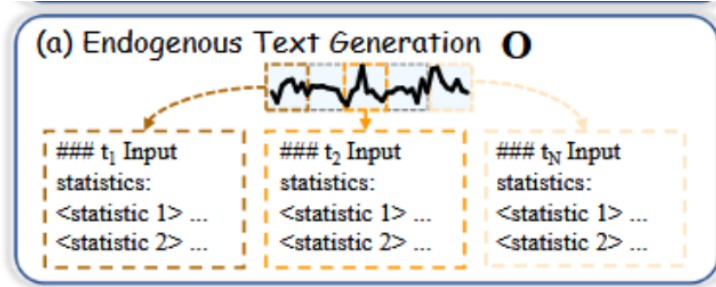
希望如何解决：利用**内容浓缩器**重构，过滤已对齐文本模态中的冗余信息。

Framework



(a) 内生文本生成。

为缓解直接转换时间序列为自然语言时的语义漂移与输出不确定性（Kowsher 等，2024；Jin 等，2023），我们设计了统一的内生文本提示模板（如**均值、极值、趋势**），并将其应用于每个分块以生成对应的内生文本。通过这种方式，可避免因生成单一全局提示所带来的限制，同时匹配时间序列的动态特性。



(b) 跨视角文本融合(Cross-view text fusion)。

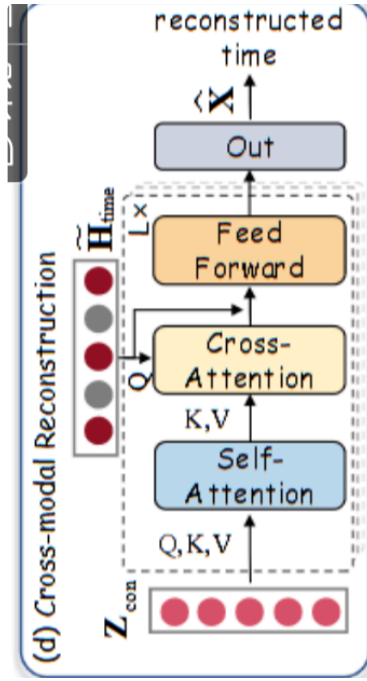
为了利用外源文本中丰富的背景知识以及内源文本与时间序列的强语义相关性，MindTS 整合了来自内源和外源文本视角的文本信息，在引入背景知识的同时实现对特定分块的精准映射。

具体而言，我们**采用了跨视角注意力机制**，能够有**选择性地**从两个文本视角中**提取互补信息**。

为了增强与时间序列的语义一致性并提取最相关的背景信息，我们以内源文本 H_{text}^O 为查询项，以外源文本 H_{text}^C 为键和值，从而得到融合后的文本表示 Z_{text} 。该过程表述为：

$$Z_{text} = \text{LayerNorm} \left(\hat{Z}_{text} + \text{FeedForward} \left(\hat{Z}_{text} \right) \right),$$

$$\hat{Z}_{text} = \text{LayerNorm} \left(H_{text}^O + \text{CrossAttn}(H_{text}^O, H_{text}^C, H_{text}^C) \right),$$



多模态对齐策略(Multimodal alignment strategy)

我们采用对比学习来显式对齐时间序列和文本两种模态，通过拉近正样本对（对齐的时间-文本）并推开负样本（无关样本），从而增强语义一致性对齐。

具体来说，两个表示 H_{time} 和 Z_{text} 之间的相似性矩阵表示如下：

$$\mathbf{K}_{TT} = \begin{bmatrix} k(\mathbf{h}_{time}^1, \mathbf{z}_{text}^1) & \dots & k(\mathbf{h}_{time}^1, \mathbf{z}_{text}^N) \\ \vdots & k(\mathbf{h}_{time}^j, \mathbf{z}_{text}^g) & \vdots \\ k(\mathbf{h}_{time}^N, \mathbf{z}_{text}^1) & \dots & k(\mathbf{h}_{time}^N, \mathbf{z}_{text}^N) \end{bmatrix} \in \mathbb{R}^{N \times N},$$

其中 $k(\cdot, \cdot)$ 表示时间与文本表征之间的相似度。若 $j = g$ ，则 $k(\mathbf{h}_{time}^j, \mathbf{z}_{text}^g)$ 被视为正样本对。因此，多模态对齐损失 L_{MA} 定义为：

$$\mathcal{L}_{MA} = -\frac{1}{2N} \left[\sum_{j=1}^N \log \frac{\exp(k(\mathbf{h}_{time}^j, \mathbf{z}_{text}^j)/\tau)}{\sum_{g=1}^N \exp(k(\mathbf{h}_{time}^j, \mathbf{z}_{text}^g)/\tau)} + \sum_{g=1}^N \log \frac{\exp(k(\mathbf{h}_{time}^g, \mathbf{z}_{text}^g)/\tau)}{\sum_{j=1}^N \exp(k(\mathbf{h}_{time}^j, \mathbf{z}_{text}^g)/\tau)} \right],$$

(c) 内容压缩器(Content condenser)

受信息瓶颈（Information Bottleneck, IB）原理启发，我们提出内容压缩器，基于对齐后的文本表征 Z_{text} 过滤冗余表示。

该过程在保留时间序列关键信息的同时，生成凝聚后的文本表征 Z_{con} 。形式上，寻找最优凝聚表征 Z_{con} 的目标定义为：

$$\mathbf{Z}_{con}^* = \arg \min_{\mathbb{P}(\mathbf{Z}_{con}|\mathbf{Z}_{text})} I(\mathbf{Z}_{text}; \mathbf{Z}_{con}) + R(\hat{\mathbf{X}}, \mathbf{Z}_{con}),$$

其中，

$l(\cdot; \cdot)$ 表示对齐后的文本表示与摘要文本表示之间的互信息。最小化该互信息促使模型学习更紧凑的表示。

互信息可以理解为： Z_{con} 从 Z_{text} 中保留了多少信息。

如果互信息很大，说明： Z_{con} 几乎保留了 Z_{text} 的全部内容。这样压缩效果就不好，因为冗余信息也被保留下来了。

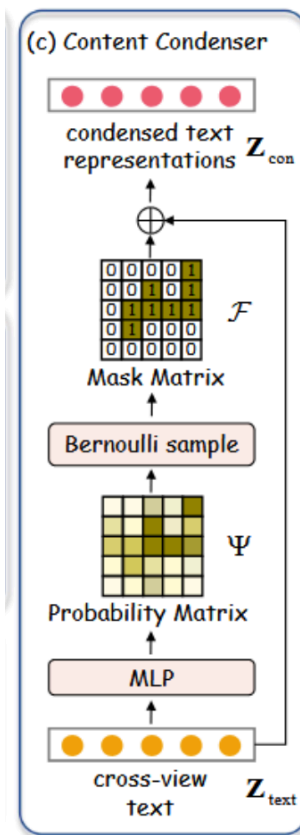
如果互信息很小，说明： Z_{con} 只保留了 Z_{text} 的一部分信息，这符合“压缩”的目标

$R(\cdot, \cdot)$ 表示重构目标函数。也就是：用压缩后的文本表示 Z_{con} 去重构时间序列 $\hat{X}$ 。基于此，我们引入跨模态重构，以确保摘要文本保留足够的信息以恢复时间序列 $\hat{X}$ 。

这里要注意：如果只最小化互信息，模型可能会走向一个极端：把所有文本信息都丢掉。这样互信息确实最小，但文本就没有任何用了。

所以必须加入重构约束，让 Z_{con} 仍然保留足够的信息，能够帮助恢复时间序列。

因此，以上公式的真实含义是：在尽量压缩文本信息的同时，还要保证压缩后的文本仍然能重构时间序列。



作者先把对齐后的文本表示 Z_{text} 输入一个 MLP，得到一个概率矩阵：

$$\Psi = [\psi_i]_{i=1}^N$$

其中每个 ψ_i 表示：第 i 个文本 patch / token / 表示单元被保留下来的概率。可以理解为一个“保留分数”。

作者根据概率 Ψ 采样一个二值 mask: $F \sim \text{Bernoulli}(\Psi)$

其中: $F_i \in \{0, 1\}$

如果: $F_i = 1$, 表示第 i 个文本单元被保留。

如果: $F_i = 0$, 表示第 i 个文本单元被遮蔽, 也就是被过滤掉。

例如: $\Psi = [0.9, 0.2, 0.8, 0.1, 0.7]$, 可能采样得到: $F = [1, 0, 1, 0, 1]$

这说明第 1、3、5 个文本单元被保留, 第 2、4 个被过滤。

作者发现, 仅仅让模型学习 mask 还不够, 因为模型可能出现两种极端情况:

极端 1: 几乎全部保留

如果模型把所有 ψ_i 都学成接近 1: $\Psi = [0.99, 0.98, 0.97, 0.99]$

那么几乎所有文本都被保留。

这时没有起到压缩作用。

极端 2: 几乎全部丢弃

如果模型把所有 ψ_i 都学成接近 0: $\Psi = [0.01, 0.02, 0.01, 0.03]$

那么文本信息几乎全部被丢掉。这时跨模态重构会变得困难。

所以作者引入一个先验分布: $G(Z_{con}) \sim \prod_{i=1}^N \text{Bernoulli}(\mu)$

这里的 $\mu \in (0, 1)$ 是一个超参数。它可以理解为: 希望平均保留多少比例的文本信息。

- $\mu = 0.8$: 倾向于保留 80% 的文本信息, 压缩较弱;
- $\mu = 0.5$: 倾向于保留 50% 的文本信息, 中等压缩;
- $\mu = 0.2$: 倾向于只保留 20% 的文本信息, 压缩较强。

所以 μ 用来控制 content condenser 的压缩强度。

另外, 用 KL 散度近似互信息

作者提出:

$$I(\mathbf{Z}_{\text{text}}; \mathbf{Z}_{\text{con}}) \leq \mathbb{E}_{\mathbf{Z}_{\text{text}}} [\text{KL}(\mathbb{P}(\mathbf{Z}_{\text{con}} | \mathbf{Z}_{\text{text}}) || \mathbb{G}(\mathbf{Z}_{\text{con}}))],$$

原本我们想最小化互信息 $I(Z_{\text{text}}; Z_{\text{con}})$, 但互信息通常不好直接计算, 所以作者用一个 KL 散度上界来近似它。

也就是说, 原目标是: $\min I(Z_{\text{text}}; Z_{\text{con}})$

但是互信息难算，于是换成： $\min KL(P(Z_{con}|Z_{text})||G(Z_{con}))$

其中：

- $P(Z_{con}|Z_{text})$ ：模型根据文本表示生成压缩表示的条件分布；
- $G(Z_{con})$ ：作者设定的先验分布；
- $KL(\cdot||\cdot)$ ：衡量两个分布之间的差异。

KL 散度越小，说明：模型学到的 mask 分布越接近预设的 Bernoulli(μ) 分布。

这就能控制压缩程度。

$$\mathcal{L}_{CC} = \sum_{i=1}^N \psi_i \log \frac{\psi_i}{\mu} + (1 - \psi_i) \log \frac{1 - \psi_i}{1 - \mu}.$$

这个公式本质上就是： $KL(Bernoulli(\psi_i)||Bernoulli(\mu))$

对每个位置 i 求和。

也就是说： $L_{CC} = \sum_{i=1}^N KL(Bernoulli(\psi_i)||Bernoulli(\mu))$

它的作用是让模型学到的保留概率 ψ_i 不要随意变化，而是整体上接近一个由 μ 控制的压缩分布。

10.1 直观理解 L_{CC}

如果 $\mu = 0.3$ ，意思是作者希望模型平均只保留约 30% 的文本信息。

那么如果某个位置的： $\psi_i = 0.3$

它和先验一致，KL 损失较小。

如果： $\psi_i = 0.95$ ，说明模型非常想保留这个位置，这会偏离先验，KL 损失变大。

如果： $\psi_i = 0.01$ ，也会偏离先验，KL 损失变大。

不过这里要注意： L_{CC} 不是唯一损失。模型还要受到跨模态重构损失约束。

所以最终效果不是简单地让所有 ψ_i 都等于 μ ，而是在重构任务的压力下：

重要文本位置可以被保留；

不重要文本位置被过滤；

整体压缩比例受到 μ 控制。

11. 为什么 L_{CC} 可以过滤冗余信息？

因为 L_{CC} 来自信息瓶颈思想，它限制 Z_{con} 从 Z_{text} 中继承过多信息。

如果没有这个限制，模型可能直接把全部文本都传过去：

$$Z_{con} \approx Z_{text}$$

这样就没有过滤作用。加入 L_{CC} 后，模型被迫控制保留信息量。

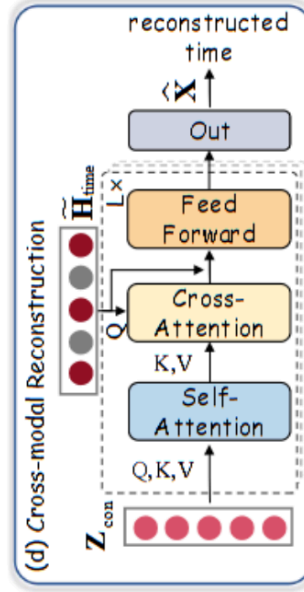
但与此同时，后面的重构任务要求模型还必须保留对时间序列有用的信息。

于是模型会倾向于保留：能帮助重构时间序列的文本内容

过滤：不能帮助重构时间序列的文本内容

这就是 content condenser 的核心机制。

(d) 跨模态重建 (Cross-modal reconstruction.)



为增强跨模态交互，一个直接的方法是从完整时间序列和精简文本中直接进行时间序列重构。

然而，由于时间序列本身已包含丰富的重构信息，这一过程无法充分促使模型捕获深层的跨模态依赖关系。为解决此问题，我们设计了一个更具挑战性的目标：

基于随机掩码时间序列 $\hat{X}$ 和精简文本 Z_{con} 进行时间序列重构。

该设计强化了跨模态依赖，并促使精简表示保留更丰富的与时间序列相关的信息。

如图 2(d)所示，对 X 进行分块与掩码处理得到 $\hat{X}$ ，随后通过时间编码器编码产生 $H_{time}^{\hat{}}$ 。该编码器与另一个时间编码器共享权重。基于输入 $H_{time}^{\hat{}}$ 和 Z_{con} ，重构输出 $\hat{X} \in R^{T \times D}$ 通过 $\hat{X} = Projection(U_{TT})$ 获得，其中 U_{TT} 定义为：

$$U_{TT} = \text{FeedForward} \left(\tilde{H}_{time} + \text{CrossAttn} \left(\tilde{H}_{time}, Z'_{con}, Z'_{con} \right) \right),$$

其中 $Z'_{con} = \text{MSA}(Z_{con}, Z_{con}, Z_{con})$ 表示自注意力层。跨模态重建函数的形式化定义为：

$$\mathcal{L}_{\text{Rec}} = \left\| \mathbf{X} - \hat{\mathbf{X}} \right\|_F^2.$$

- 文章用了哪些实验数据集

- 文章使用了 6 个真实世界多模态时间序列异常检测数据集：Weather、Energy、Environment、KR、EWJ、MDT。每个数据集都包含两类信息：一类是数值时间序列，另一类是与时间序列对应的外生文本信息。这些数据覆盖 天气、能源、环境、金融 4 个领域。

- Weather

- ◆ 领域：天气/气象。
- ◆ 维度为 4，异常比例为 17.10%，平均总长度为 12339，时间跨度为 2012.7.17 – 2023.10.20。
- ◆ 数据包括从政府网站收集的**温度、湿度统计数据**以及相关报告。这个数据集用于验证模型是否能利用气象文本背景帮助识别天气时间序列中的异常变化。

- Energy

- ◆ 领域：能源。
- ◆ 维度为 9，异常比例为 17.23%，平均总长度为 1622，时间跨度为 1993.4.5 – 2024.4.29。
- ◆ 数据包括**汽油价格统计数据**以及来自 U.S. Energy Information Administration 的能源报告。外生文本描述市场需求、经济周期等背景因素，用来辅助检测能源价格时间序列中的异常。

- Environment

- ◆ 领域：环境/空气质量。
- ◆ 维度为 1，异常比例为 5.81%，平均总长度为 15981，时间跨度为 1980.1.1 – 2023.9.30。
- ◆ 数据包括 Air Quality Index 空气质量指数，以及来自 U.S. Environmental Protection Agency 和 NBC 的相关报告。这个数据集用于检测环境监测序列中的异常空气质量变化。

- KR

- ◆ 领域：金融。
- ◆ 维度为 1，异常比例为 6.21%，平均总长度为 2655，时间跨度为 2009.9.15 – 2020.4.1。
- ◆ 数据包括来自 Yahoo 的股票数值数据，以及相关金融文本信息。用于验证模型是否能结合股价时间序列和金融新闻背景检测市场异常。

- EWJ

- ◆ 领域：金融。
- ◆ 维度为 1，异常比例为 9.96%，平均总长度为 2658，时间跨度为 2009.11.16 – 2020.6.9。
- ◆ 数据包括金融时间序列以及来自多个金融新闻网站的新闻信息。用于评估模型在金融市场事件驱动异常检测中的效果。

- MDT

- ◆ 领域：金融。

- ◆ 维度为 1，异常比例为 11.17%，平均总长度为 2732，时间跨度为 2009.8.6 – 2020.6.12。
- ◆ 数据包括金融数值数据和来自 NASDAQ、Bloomberg 等金融新闻网站的文本信息。这个数据集同样用于检验金融时间序列和新闻文本的多模态异常检测能力。

○ 数据集构建细节：

- ◆ 作者说明这些数据集都是公开可用的真实世界数据集，不包含个人身份信息。
- ◆ 外生文本通过网页搜索和定向爬取获得，包括新闻、官方报告、机构报告等。
- ◆ 为了保证文本与时间序列具有语义相关性，作者为每个数据集定义了 2–3 个领域关键词，并收集排名靠前的搜索结果。
- ◆ 每条文本都保留 timestamp、source、title、content 等结构化信息。
- ◆ 为了防止未来信息泄漏，作者做了两层处理：
 - 第一，所有文本必须有明确时间戳，避免未来文本匹配到过去观测值；
 - 第二，将文本拆分为历史事实描述和预测性描述，只保留事实内容，避免未来预测信息泄漏。
- ◆ 文本与时间序列之间采用起止日期标记的时间对齐方式，但作者强调 MindTS 不严重依赖严格时间对齐，也可以在 window-level matching 场景下工作。

• 实验对比了什么 Baseline

- 主实验中，作者将 MindTS 与 17 个竞争性 baseline 进行比较；附录 A.2 中进一步列出 19 个 baseline，额外包括 CALF 和 DualTF。主表 Table 1 和 Table 2 主要报告的是 17 个 baseline 的异常检测结果。

○ LLM-based methods

- ◆ LLMixer / LMixer (Kowsher et al.–24)：基于大语言模型的时间序列分析方法，将时间序列进行多尺度分解，并将多尺度信号和文本提示输入预训练 LLM，从而利用 LLM 的语义知识进行时间序列建模。
- ◆ UniTime / UTime (Liu et al.–24c)：基于提示学习的大语言模型时间序列方法，引入可学习 prompt、prompt pool 和领域特定指令，从 LLM 中激发领域相关的时间知识。
- ◆ GPT4TS / G4TS (Zhou et al.–23)：基于 GPT 的时间序列方法，通过选择性微调位置编码、LayerNorm 等关键组件，使预训练语言模型能够适配时间序列任务。
- ◆ CALF (Liu et al.–25)：附录中列出的 LLM-based baseline，提出跨模态微调框架，用于缓解时间预测分支和对齐文本源分支之间的分布差异，从而增强跨模态对齐能力。

○ Pre-trained methods

- ◆ DADA (Shentu et al.–25)：通用时间序列异常检测预训练模型，在广泛领域上进行预训练，可适配不同下游异常检测任务。
- ◆ Timer (Liu et al.–24d)：将异构时间序列统一成单一序列，并通过序列建模方式进行预测式异常检测。

- ◆ UniTS (Gao et al.-24): 将多个时间序列任务转化为统一 token 表示, 并使用 prompt-based 框架; 在异常检测中生成 masked token, 并利用去噪输出识别异常。

- Deep learning-based methods

- ◆ ModernTCN / Modern (Luo & Wang-24): 纯卷积时间序列架构, 用于解耦并建模时间关系、通道关系和变量关系。
- ◆ TimesNet / TsNet (Wu et al.-23): 将复杂时间模式分解到不同频率成分, 并将一维时间序列映射到二维空间, 以同时建模周期内和周期间动态。
- ◆ DCdetector / DC (Yang et al.-23): 基于对比学习的时间序列异常检测方法, 从 patch-wise 和 point-wise 两个视角区分正常模式和异常模式。
- ◆ Anomaly Transformer / A.T. (Xu et al.-21): 基于异常点更倾向于与邻近时间点产生关联的假设, 使用 minimax 策略放大关联差异, 从而增强异常判别能力。
- ◆ PatchTST / Patch (Nie et al.-22): 基于 patch 的时间序列 Transformer 方法, 采用通道独立 patching, 提高模型捕捉局部时间特征的能力。
- ◆ TranAD (Tuli et al.-22): 基于 Transformer 的异常检测模型, 通过对抗训练放大重构误差, 从而增强异常检测效果。
- ◆ DualTF (Nam et al.-24): 附录中列出的 deep learning baseline, 采用时间域和频率域双分支结构, 通过嵌套滑动窗口分别处理外层时间域和内层频率域, 并对齐二者的异常分数。
- ◆ iTransformer / iTrans (Liu et al.-23): 将时间信息嵌入变量 token, 并使用 attention 机制建模多变量相关性。

- Non-learning methods

- ◆ PCA (Shyu et al.-03): 通过样本在主成分空间中的偏离程度检测异常, 假设异常点远离正常数据分布。
- ◆ IForest / IF (Liu et al.-08): 孤立森林方法, 通过递归划分显式隔离异常点, 而不是直接建模正常行为。
- ◆ LODA (Pevný-16): 通过多个一维直方图近似联合分布, 用于识别离群点。
- ◆ HBOS (Goldstein & Dengel-12): 基于直方图的无监督异常检测方法。

- 除了单模态 baseline, 作者还做了一个重要对比:

- ◆ 将 11 个表现较好的近期方法扩展到 MM-TSFLib 多模态时间序列框架中, 得到 DADA*、LMixer*、UTime*、Timer*、UniTS*、G4TS*、Modern*、TsNet*、Patch*、TranAD*、iTrans*。
- ◆ MM-TSFLib 通过对时间序列模型输出和基于 bag-of-words 的文本嵌入进行线性插值来整合文本信息。
- ◆ 作者用这个实验说明: 即使把已有强 baseline 扩展成多模态版本, MindTS 仍然取得最好或最具竞争力的结果。

- 实验具体有哪些细节

- 是否开源

- ◆ 文章明确给出了代码地址：
- ◆ <https://github.com/decisionintelligence/MindTS>

- 实现设备

- ◆ 实验使用 **PyTorch** 实现。
- ◆ 训练和测试使用 **NVIDIA Tesla-A800-80GB GPUs**。
- ◆ 所有 baseline 都在相同硬件上由作者重新运行，以保证公平比较。

- 评估协议

- ◆ 作者遵循 **TFB** 的评估协议。
- ◆ 测试阶段禁用 **drop last** 操作，避免由于最后一个 batch 被丢弃造成不公平比较。
- ◆ 所有 baseline 使用官方或 GitHub 开源实现，并遵循各自论文推荐配置。

- 优化器

- ◆ 使用 **Adam optimizer** 训练。

- Batch size

- ◆ 初始 batch size 设置为 **64**。
- ◆ 如果出现 **Out-Of-Memory**，batch size 会减半。
- ◆ 最小 batch size 为 **8**。

- 损失函数与权重

- ◆ MindTS 的总损失由三部分组成：

$$\begin{aligned} \backslash[& L = L_{\{MA\}} + L_{\{CL\}} + L_{\{Rec\}} \\ \backslash] \end{aligned}$$

- ◆ 其中：

- L_{MA} ：多模态时间-文本语义对齐损失；
- L_{CL} ：内容压缩器损失，包括 L_{CC} 和 L_{SM} ；
- L_{Rec} ：跨模态时间序列重构损失。

- ◆ 作者设置所有优化目标的权重都为 **1**。
- ◆ 也就是说，论文没有额外手工调整三个损失项之间的权重比例。

- 训练参数

- ◆ 训练中会更新 MindTS 中的时间序列编码器、文本编码器相关表示层、跨视角文本融合模块、多模态对齐模块、内容压缩器、跨模态重构模块等参数。
- ◆ 时间序列输入先经过 **instance normalization** 和 **channel-independent processing**。
- ◆ 时间序列被切分为 patch：

\[\quad P = \text{Patching}(X)

\]

- ◆ 每个 patch 的大小为 p ，滑动步长为 l ，patch 总数为：

\[\quad N = \lceil T-p \rceil / l + 1

\]

- ◆ 每个 patch 会生成对应的内生文本，用于细粒度时间-文本对齐。

○ 关键超参数

- ◆ **patch size p** ：

- 作者在参数敏感性实验中测试了不同 patch size。
- 实验发现，patch size 增大时，模型性能通常先提升后下降。
- patch size 较小时显存开销更大。
- 作者说明实验中 patch size 通常设置为 6。

- ◆ **mask ratio m** ：

- 用于随机遮蔽时间序列，构造 masked time series $\tilde{X}$ 。
- 作者发现 mask ratio 接近 50% 时通常表现较好。
- 当 mask ratio 继续增大时，重构原始时间序列变得更困难，模型性能下降。

- ◆ **compression strength μ** ：

- 用于控制 content condenser 的文本压缩程度。
- 作者测试了 $\mu \in [0.1, 0.9]$ 的范围。
- 实验显示模型在较大范围内都能保持较高性能，说明 content condenser 对不同压缩强度较鲁棒。

- ◆ **temperature τ** ：

- 出现在多模态对比学习损失 L_{MA} 中，用于控制 time-text 相似度 softmax 的分布尖锐程度。
- 论文给出了符号和作用，但没有在正文或附录实现细节中报告具体取值。

○ LLM 使用细节

- ◆ 作者默认使用 **DeepSeek** 作为 MindTS 中的 LLM。
- ◆ LLM 的作用主要有两个：
 - 作为文本编码器提取文本特征；
 - 用于根据原始时间序列 patch 生成内生文本。
- ◆ 作者声明没有使用 LLM 辅助论文写作。
- ◆ 作者额外比较了 **GPT2**、**BERT**、**LLAMA**、**DeepSeek**，结论是 MindTS 对具体 LLM 选择不敏感，性能提升主要来自模型结构设计，而不是依赖某一个特定 LLM。

○ 论文没有报告的实现细节

- ◆ 文章没有明确给出 learning rate。
- ◆ 没有明确给出训练 epoch 数。
- ◆ 没有明确给出 hidden dimension、Transformer 层数、attention head 数。
- ◆ 没有明确给出 temperature τ 的具体数值。
- ◆ 没有明确给出每个数据集的 train/validation/test 划分比例。

Datasets	Metric	MindTS	DADA	LMixer	UTime	Timer	UniTS	G4TS	Modern	TsNet	DC	A.T.	Patch	TranAD	iTrans	PCA	IF	LODA	HBOS
Weather	Aff-F	82.66	69.01	73.68	76.46	75.46	76.17	72.56	<u>81.06</u>	80.58	42.80	49.22	77.17	77.81	75.37	64.91	54.06	52.55	47.70
	V-PR	57.48	30.00	43.47	51.90	43.21	44.35	41.30	52.13	50.09	18.33	19.17	50.03	52.08	42.56	47.13	49.66	<u>55.03</u>	46.58
	V-ROC	82.64	61.03	71.71	78.45	73.22	75.08	70.03	81.14	<u>81.91</u>	45.56	43.32	79.97	78.75	73.37	57.38	56.45	57.00	54.16
Energy	Aff-F	74.37	64.38	65.85	61.98	60.20	63.84	66.37	70.76	66.00	47.07	43.39	66.85	49.69	70.81	57.65	62.03	63.45	55.85
	V-PR	50.36	34.18	30.35	32.88	29.46	31.04	31.68	36.60	38.61	22.57	19.69	34.41	33.80	35.82	44.30	46.03	48.63	42.57
	V-ROC	74.44	54.37	53.04	49.97	46.03	51.15	53.10	<u>65.05</u>	59.47	45.93	31.56	58.31	56.37	63.06	53.07	53.61	55.90	51.50
Environment	Aff-F	85.29	84.11	84.36	81.71	84.19	83.06	72.26	81.07	80.41	62.24	59.75	81.17	61.41	74.43	55.63	46.50	46.45	22.25
	V-PR	56.79	54.20	52.94	48.87	51.42	50.24	23.94	42.26	50.64	7.69	18.14	45.78	4.91	24.87	17.87	8.94	18.66	52.15
	V-ROC	93.78	87.69	89.75	91.77	<u>92.10</u>	92.03	66.79	89.78	87.97	41.28	51.98	90.86	14.20	73.81	37.08	46.20	50.69	51.03
KR	Aff-F	90.28	84.22	71.80	88.58	<u>89.55</u>	82.24	79.56	84.42	85.47	61.94	70.99	79.52	73.26	79.49	58.11	69.38	60.96	64.78
	V-PR	53.15	45.90	15.13	46.87	51.41	43.32	38.23	39.95	51.60	8.48	7.94	36.18	28.42	27.37	24.19	43.31	51.82	<u>52.06</u>
	V-ROC	89.86	70.82	52.79	73.55	75.99	73.93	67.81	<u>88.87</u>	79.00	43.04	41.97	74.65	41.05	76.12	47.51	60.70	59.99	61.41
EWJ	Aff-F	83.89	81.26	66.86	78.22	78.06	77.61	76.65	81.57	81.82	48.10	59.03	75.82	69.22	78.27	51.06	67.55	72.06	71.03
	V-PR	50.42	43.36	15.21	32.39	33.17	39.32	35.63	44.75	43.15	15.37	10.85	36.08	17.80	28.98	19.38	37.81	40.08	41.19
	V-ROC	84.12	71.79	46.80	64.49	67.72	73.91	67.95	<u>83.88</u>	75.76	47.10	31.75	71.56	49.60	72.16	45.26	59.24	61.65	62.07
MDT	Aff-F	89.19	77.99	67.65	76.28	78.51	75.57	<u>80.81</u>	<u>80.81</u>	80.08	47.33	66.12	79.47	63.93	78.66	54.66	53.74	55.06	52.33
	V-PR	65.44	46.81	19.10	38.94	38.38	37.61	44.81	<u>52.18</u>	50.53	15.72	15.93	41.67	14.34	36.36	22.93	35.32	44.63	44.77
	V-ROC	83.02	66.76	47.06	61.00	60.28	58.67	62.30	<u>82.30</u>	79.56	45.02	44.53	77.69	28.55	71.87	44.09	54.02	55.98	55.30

Table 5: Ablation study on $\mathcal{L}_{SM}$ (all results in %, best results are highlighted in bold).

Method	Weather			Energy			Environment			KR			EWJ			MDT		
Metric	Aff-F	V-PR	V-ROC	Aff-F	V-PR	V-ROC	Aff-F	V-PR	V-ROC	Aff-F	V-PR	V-ROC	Aff-F	V-PR	V-ROC	Aff-F	V-PR	V-ROC
MindTS (w/o $\mathcal{L}_{SM}$)	81.59	55.88	81.48	73.26	48.53	73.18	83.40	54.19	91.58	88.49	49.20	88.31	82.57	49.25	83.64	86.29	63.23	81.11
MindTS	82.66	57.48	82.64	74.37	50.36	74.44	85.29	56.79	93.78	90.28	53.15	89.86	83.89	50.42	84.12	89.19	65.44	83.02